



คู่มือการนำระบบขึ้นใช้งานจริง (Production Deployment Guide)

ระบบบริหารจัดการโครงการองค์กร (AG Projects Enterprise — Zoho Projects Clone)



ข้อมูลจำเพาะทางเทคนิคของระบบ (System Architecture)

ก่อนนำระบบขึ้นสู่เซิร์ฟเวอร์จริง (Production) ควรทำความเข้าใจโครงสร้างของแอปพลิเคชัน:

- Backend:** Node.js (Express.js) รันบนพอร์ตมาตรฐาน `3000`
- Database:** SQLite 3 เก็บข้อมูลในไฟล์ `zoho_tracker.db` (แบบ Local File Database)
- File Storage:** เก็บไฟล์แนบงานต่างๆ ในโฟลเดอร์ `uploads/`
- Authentication:** รองรับทั้ง Local Email/Password และ Microsoft OAuth (MSAL.js / Azure AD)
- Frontend:** Pure HTML5, Vanilla CSS (Design System), and Vanilla JavaScript (SPA)



ข้อควรระวังสำคัญที่สุดสำหรับระบบ SQLite (Critical Warning)

เนื่องจากระบบใช้ฐานข้อมูล SQLite ซึ่งบันทึกข้อมูลทั้งหมดลงในไฟล์ `zoho_tracker.db` บนเครื่องเซิร์ฟเวอร์โดยตรง:

- ห้าม Deploy บน Serverless Cloud แบบไร้ Disk (เช่น Vercel, Netlify, AWS Lambda, Heroku แบบไม่ต่อ Disk) เนื่องจากเมื่อ Serverless ทำการ Restart หรือ Sleep ไฟล์ข้อมูลทั้งหมดจะหายไป (Data Loss)!
- วิธีที่ถูกต้อง: ต้อง Deploy บน VPS (Virtual Private Server), Dedicated Linux Server หรือใช้ Docker Container ที่มีการต่อ Mounted Volume / Persistent Disk เท่านั้น



วิธีที่ 1: การ Deploy บน VPS Linux (Ubuntu 22.04 / 24.04 LTS) ด้วย PM2 และ Nginx

🌟 (แนะนำที่สุดสำหรับองค์กร — เสถียรที่สุดและจัดการข้อมูล SQLite ได้ปลอดภัย 100%)

สามารถใช้ผู้ให้บริการ VPS ชื่อนำ เช่น DigitalOcean, AWS EC2, Google Cloud Compute Engine, Linode, Hostinger VPS หรือเครื่องเซิร์ฟเวอร์ภายในบริษัท (On-Premises)

1.1 เตรียมความพร้อมเซิร์ฟเวอร์และติดตั้ง Node.js

เข้าใช้งานเซิร์ฟเวอร์ผ่าน SSH แล้วรันคำสั่งเพื่ออัปเดตระบบและติดตั้ง Node.js LTS (v20 หรือ v22):

```
# อัปเดตรายการแพ็คเกจ
```

```
sudo apt update && sudo apt upgrade -y
```

```
# ติดตั้ง Curl และ Git
```

```
sudo apt install curl git build-essential -y
```

```
# ติดตั้ง Node.js v20 LTS ผ่าน NodeSource
```

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

```
# ตรวจสอบเวอร์ชัน (ควรได้ v20.x และ npm v10.x ขึ้นไป)
```

```
node -v
```

```
npm -v
```

1.2 ดาวน์โหลดและติดตั้งโค้ดแอปพลิเคชัน

สร้างโฟลเดอร์สำหรับแอป และนำโค้ดขึ้นไปบนเซิร์ฟเวอร์ (ผ่าน Git Clone หรือ SFTP):

```
# สร้างโฟลเดอร์ใน /var/www
```

```
sudo mkdir -p /var/www/ag-projects
```

```
sudo chown -R $USER:$USER /var/www/ag-projects
```

```
cd /var/www/ag-projects
```

```
# คัดลอกโคดมาวาง หรือ Git Clone
```

```
# git clone https://github.com/your-org/ag-projects.git .

# ติดตั้ง Dependencies เฉพาะตัวที่ใช้ใน Production
npm ci --only=production

# สร้างโฟลเดอร์อัปโหลดไฟล์และตั้งสิทธิ์ให้ระบบอ่านเขียนได้
mkdir -p uploads
chmod 775 uploads
```

1.3 ติดตั้งและจัดการ Process ด้วย PM2

PM2 คือ Process Manager ช่วยให้ Node.js ทำงานตลอดเวลา (24/7) แอมทำงานเบื้องหลัง และ Restart อัตโนมัติหากเซิร์ฟเวอร์รีบูตหรือแอปสะดุด

```
# ติดตั้ง PM2 แบบ Global
sudo npm install -g pm2

# เริ่มทำงานแอปพลิเคชันพร้อมตั้งชื่อโปรเซสว่า "ag-projects"
pm2 start server.js --name "ag-projects" --time

# สั่งให้ PM2 จดสถานะและตั้งค่าให้เริ่มทำงานอัตโนมัติเมื่อ Boot เซิร์ฟเวอร์ใหม่
pm2 startup
pm2 save

# คำสั่งสำคัญสำหรับดูแลระบบด้วย PM2
pm2 status          # ดูสถานะการทำงาน
pm2 logs ag-projects # ดู Log แบบ Real-time
pm2 restart ag-projects # สั่งรีสตาร์ทแอป
```

1.4 ตั้งค่า Nginx Reverse Proxy และ HTTPS (SSL ฟรีจาก Let's Encrypt)

เราจะใช้ **Nginx** เป็นด่านหน้าในการรับส่งข้อมูลพอร์ต 80 (HTTP) และ 443 (HTTPS) แล้วส่งต่อมาให้ Node.js ที่พอร์ต 3000:

```
# ติดตั้ง Nginx และ Certbot (สำหรับทำ SSL Free)
sudo apt install nginx certbot python3-certbot-nginx -y
```

สร้างไฟล์ Config ของ Nginx สำหรับโดเมนของคุณ (เช่น `projects.yourdomain.com`):

```
sudo nano /etc/nginx/sites-available/ag-projects
```

นำค่าคอนฟิกนี้ไปวาง (เปลี่ยน `projects.yourdomain.com` เป็นชื่อโดเมนจริงของคุณ):

```
server {
    listen 80;
    server_name projects.yourdomain.com;

    client_max_body_size 50M; # รองรับการอัปโหลดไฟล์แนบสูงสุด 50MB

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

เปิดใช้งานไซต์และตรวจสอบความถูกต้อง:

```
# เปิดใช้งาน Config
sudo ln -s /etc/nginx/sites-available/ag-projects /etc/nginx/sites-enabled/

# ตรวจสอบว่า Syntax ถูกต้องหรือไม่
```

```
sudo nginx -t
```

```
# รีสดาร์ท Nginx
```

```
sudo systemctl reload nginx
```

```
# ติดตั้ง SSL Certificate ฟรี (Let's Encrypt HTTPS)
```

```
sudo certbot --nginx -d projects.yourdomain.com
```

(ระบบจะถามอีเมล ให้ตอบตกลง และเลือกให้ Redirect HTTP -> HTTPS อัตโนมัติ)



วิธีที่ 2: การ Deploy ด้วย Docker & Docker Compose (Modern Container setup)

✨ (เหมาะสำหรับการนำไปรันบน Cloud Platforms เช่น AWS ECS, Google Cloud Run, Coolify, Portainer, หรือ Server ที่ใช้ Docker)

ในโครงสร้างโปรเจกต์ เราได้เตรียมไฟล์ `Dockerfile` , `.dockerignore` , และ `docker-compose.yml` ไว้ให้พร้อมใช้งานทันทีครับ!

2.1 โครงสร้างไฟล์ Docker ที่เตรียมไว้ให้

- `Dockerfile` : ใช้ฐานจาก `node:20-alpine` นำหนักเบา พร้อมติดตั้งไลบรารีที่จำเป็นสำหรับ SQLite
- `docker-compose.yml` : มีการตั้งค่า **Persistent Volume** (`./zoho_tracker.db:/app/zoho_tracker.db` และ `uploads_data`) ทำให้เมื่อหยุดหรือรีสตาร์ท Container ข้อมูลในฐานข้อมูลไฟล์จะไม่หายไป!

2.2 ขั้นตอนการรันด้วย Docker Compose บนเครื่อง Server

บนเครื่องเซิร์ฟเวอร์ที่มี Docker และ Docker Compose ติดตั้งอยู่แล้ว:

```
# 1. เข้าไปที่โฟลเดอร์โปรเจกต์
```

```
cd /path/to/zoho-project-tracker
```

```
# 2. สร้างไฟล์ฐานข้อมูลเปล่าไว้ก่อน (หากยังไม่มี) เพื่อให้ Docker Mount ไม่เกิด Error
```

```
touch zoho_tracker.db
```

```
# 3. สั่ง Build และ Run Container แบบ Background
```

```
docker compose up -d --build
```

```
# 4. ตรวจสอบสถานะว่า Container ขึ้นรันปกติหรือไม่
```

```
docker compose ps
```

```
docker compose logs -f
```

ระบบจะเปิดพอร์ต `3000` ให้เข้าใช้งานทันที! สามารถเอา Nginx มาคลุมด้านหน้าเพื่อต่อ HTTPS ได้ตามขั้นตอน 1.4 ครับ



วิธีที่ 3: การ Deploy บน Cloud Platform บริการสำเร็จรูป (Railway / Render / Fly.io)

หากไม่ต้องการจัดการ Linux Server เอง สามารถใช้ PaaS (Platform as a Service) ได้ แต่ต้องตั้งค่า **Persistent Disk / Volume** เสมอ!



การตั้งค่าบน Railway.app (แนะนำสำหรับ PaaS)

- อัปโหลดโค้ดขึ้น GitHub Repository
- เข้าสู่เว็บ **Railway.app** กด **New Project -> Deploy from GitHub repo**
- ไปที่เมนู **Variables** เพิ่มตัวแปรสภาพแวดล้อม:
 - `NODE_ENV` = `production`
 - `PORT` = `3000`
- 🔥 **ขั้นตอนสำคัญที่สุด (ป้องกันข้อมูลหาย):**
 - ไปที่แท็บ **Volumes** กด **+ New Volume**
 - ตั้งชื่อ Volume ว่า `sqlite-db-volume`
 - ตั้งค่า **Mount Path** ให้เป็น `/app/data` (หรือ `/app/uploads` ถ้าต้องการเก็บรูป)
- ปรับแก้ path ในโค้ดหรือตารางให้เชื่อมกับ Volume และกด **Deploy** ระบบจะให้โดเมน `https://xxx.up.railway.app` พร้อมใช้งานทันทีด้วย HTTPS!

🔒 4. เช็กลิสต์ความพร้อมก่อนใช้งานจริง (Production Readiness Checklists)

4.1 ตั้งค่า Microsoft OAuth (Azure AD / Entra ID) ในระบบจริง

หากองค์กรต้องการใช้ปุ่ม "Sign in with Microsoft":

- เข้าไปที่เว็บ **Azure Portal** (portal.azure.com) -> ไปที่บริการ **Microsoft Entra ID**
- เลือกเมนู **App registrations** -> เลือกแอปที่ลงทะเบียนไว้ (หรือกด New registration)
- ในแท็บ **Authentication** (การรับรองตัวตน) -> หัวข้อ **Redirect URIs**:
 - เพิ่ม URL โดเมนจริงของคุณ เช่น `https://projects.yourdomain.com` (ต้องมี `https://` เสมอใน Production)
- คัดลอก **Application (client) ID** มาใส่ในไฟล์ `public/app.js` บรรทัดที่ 7:

```
const MS_CLIENT_ID = '12345678-abcd-ef01-2345-67890abcdef0'; // ใส่ Client ID จริงขององค์กร
```

4.2 ตั้งค่าการตั้งเวลาสำรองข้อมูลฐานข้อมูลอัตโนมัติ (Automated DB Backup Cron Job)

ป้องกันเหตุสุดวิสัย (Disk พัง, โดนแฮก, ลบผิด) ควรตั้งระบบสำรองไฟล์ `zoho_tracker.db` ทุกเที่ยงคืน:

```
# สร้างโฟลเดอร์สำหรับเก็บไฟล์ Backup
sudo mkdir -p /var/backups/ag-projects
sudo chmod 700 /var/backups/ag-projects
```

```
# ตั้งเวลาทำงานอัตโนมัติด้วย Crontab
sudo crontab -e
```

ใส่บรรทัดนี้ไว้ด้านล่างสุดของไฟล์ Crontab (สำรองข้อมูลทุกเที่ยงคืนและเก็บย้อนหลัง 30 วัน):

```
0 0 * * * cp /var/www/ag-projects/zoho_tracker.db /var/backups/ag-projects/zoho_tracker_$(date +%Y%m%d).db && find /var/backups/ag-projects
```

4.3 ระบบพิมพ์คู่มือ PDF (PDF Generation Dependencies)

ในระบบเรามีสคริปต์ `generate_pdf.cjs` สำหรับสร้างคู่มือ PDF อัตโนมัติด้วย Edge/Chrome Headless หากต้องการรันคำสั่งนี้บน Linux Server ให้ติดตั้ง Chromium เพิ่มเติมด้วย:

```
# สำหรับ Ubuntu / Debian
sudo apt install chromium-browser -y
```

📞 สรุปภาพรวมคำแนะนำ

สำหรับองค์กรธุรกิจทั่วไป แนะนำให้เลือก **วิธีที่ 1 (VPS Linux + PM2 + Nginx)** หรือ **วิธีที่ 2 (Docker Compose บน VPS)** ครับ เพราะให้ประสิทธิภาพสูงสุด ควบคุมความปลอดภัยได้ 100% และไม่มีค่าใช้จ่ายแฝงเรื่อง Disk I/O ของ Cloud PaaS ครับ 🚀